LAB: REVERSE ENGINEERING CAN BUS TRAFFIC WITH ICSIM

MADE BY: KYRYL YERMOCHENKO | STUDENT OF MICAC [ARQUS University Alliance]
DATE: 10.10.2026

SCOPE: This document records an educational experiment. The specific data provided in the work is the result of experiments I conducted with the software listed below.

1. Objective and Test Environment

Characterize the traffic on the virtual CAN bus, separating the background traffic that passes through the bus (sensors operations, communication between ECUs units in idle state, etc) from the specifically targeted, manually and automatically triggered signals.

Environment: Ubuntu/WSL, SocketCAN virtual interface vcan0, ICSim (instrument cluster and controls), can-utils, Python 3.

Simulator source: zombieCraig/ICSim.

Setup and inspection commands

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
./icsim vcan0          # cluster sim. window
./controls vcan0     # control sim. window
candump vcan0     # traffic sanity check
```

For the replay experiments, the controls process was stopped from time to time so that recorded traffic, rather than manual input, drove the cluster.

2. Traffic Capture & Baseline Analysis

Capture commands

```
candump -L vcan0 > idle1.log
candump -L vcan0 > idle2.log
candump -L vcan0 > left.log
candump -L vcan0 > right.log
```

The first steps of a practical task includes recording two periods of the can bus in idle state in order to get statistical info about communications that occur at the background of attacker future activity.

Although, it is fine to use even one recording, in this situation we are using two: to split recording process into smaller parts. The reason is quite simple — we want to capture CAN bus traffic in two periods with gap in time to check if any potential activity may be in place after some significant delay.

After collecting the necessary data, you should compare the results using automated methods. In our case, this is a Python script [Script 1] called idstats.py.

First, we're interested in:

Counting messages by CAN ID, storing timestamps, calculating median/minimum/maximum inter-arrival intervals, flagging intervals longer than twice the median, and counting distinct payloads per ID.

Metric	Idle1.log	Idle2.log
CAN frames	21,041	22,709
Unique CAN IDs	36	36
Most variable ID	0x244 (248 payloads)	0x244 (249 payloads)
Second-most variable	0x183 (64 payloads)	0x183 (64 payloads)
0x158 median interval	10.054 ms	10.036 ms
0x095 median interval	10.048 ms	10.042 ms
0x244 median interval	15.442 ms	15.437 ms

The same set of 36 CAN IDs was present in both recordings during idle. Many IDs clustered around median intervals of about 10, 20, 30, 40, 60, 100, 300, 600, or 1000 ms, which aligns with periodic scheduling. However, large maximum delays and variability prevent considering just the medians as proof of strict periodicity.

The two idle recordings differed in their individual data sets only for ID 0x244: 7 data points appeared only in idle2 and 6 only in idle1. This gives a baseline of naturally variable values that can be compared with stimuli. Overall frame counts shouldn't be taken as changes in traffic rate without normalizing for recording duration.

3. Differential Analysis and Live Inspection

Analysis and Inspection commands

```
python3 diffids.py idle1.log idle2.log
python3 diffids.py idle1.log left.log
python3 diffids.py idle1.log right.log
cansniffer -c vcan0
grep '188#' left.log | head -n 10
```

Baseline: 36 common IDs, none unique to any idle state; only ID 0x244 changed its payload values.
Left stimulus: 0x188 added payload 01000000; 0x244 also changed.
Right stimulus: 0x188 added 02000000; 0x244 changed, and 0x183 didn't have a single base value.

These changes show 0x188 as the most likely candidate for the indicator, while variations of 0x244 are present even in the idle state.

CAN ID / state	Observed four-byte data	Evidence
0x188 / idle	00000000	Idle and non-active phases
0x188 / left	01000000	Appears in the left stimulus capture
0x188 / right	02000000	Appears in the right stimulus capture

During real-time analysis, I was able to detect that the load from CAN ID 0x188 changed during turn signal operation tests. The obtained payload creates a good theory to check in [Part 5].

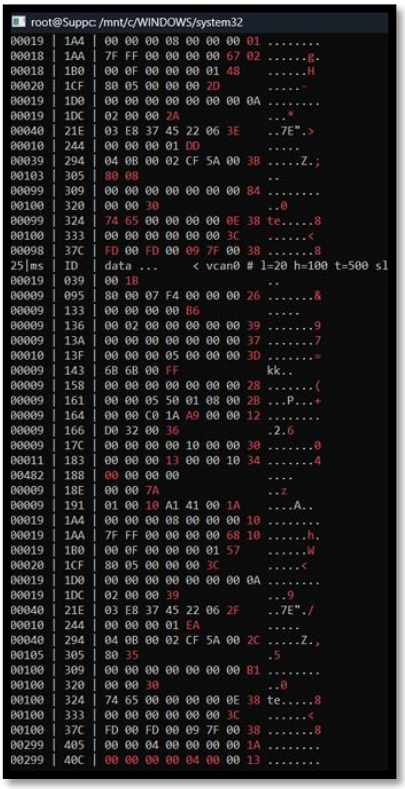


Figure 1: Live CAN traffic inspection with cansniffer; ID 0x188 appears among the observed IDs.

4. Replay-Based ID Bisection

Replay and filtering commands

```
canplayer -I left.log vcan0=vcan0
grep ' 188#' left.log > left_188.log
canplayer -I left_188.log vcan0=vcan0
python3 filterlog.py left.log left_188_python.log 188
cmp left_188.log left_188_python.log
```

A rerun with the full log and a run just with ID-0x188 both triggered the left indicator on the ICSim dashboard. The log generated by Python only for 0x188 had 46 frames and completely matched the log obtained via grep

(cmp showed no differences). The re-recorded left.log had 46,134 frames; it replaced the previous recording of the left indicator stimulus after I accidentally redirected the shell on my side.

One consolidated bisection table

“Next retained set” records the group that was tracked on the way to 0x188.

Round	Subset replayed / IDs	Frames	Effect	Next retained set
0	Full left.log	46, 134	✓	36 IDs
1	A / 18	38,526	✓	A
1	B / 18	7,608	✗	A
2	A1 / 9	22,152	✗	A2
2	A2 / 9	16,374	✓	A2
3	A21 / 4 [164 166 17C 183]	9,711	✗	A22
3	A22 / 5 [188 18E 191 1A4 1AA]	6,663	✓	A22
4	A221 / 2 [188 18E]	1,389	✓	A221
4	A222 / 3 [191 1A4 1AA]	5,274	✗	A221
5	A2211 / 1 [188]	46	✓	0X188
5	A2212 / 1 [18E]	1,343	✗	0X188

Результат: The progressive grouping reached group_A2211.log, which contains only 0x188 (46 frames). Playing it activated the left turn signal. This confirms that messages with CAN ID 0x188 are enough to control this visible function in the simulated environment, without the other 35 IDs.

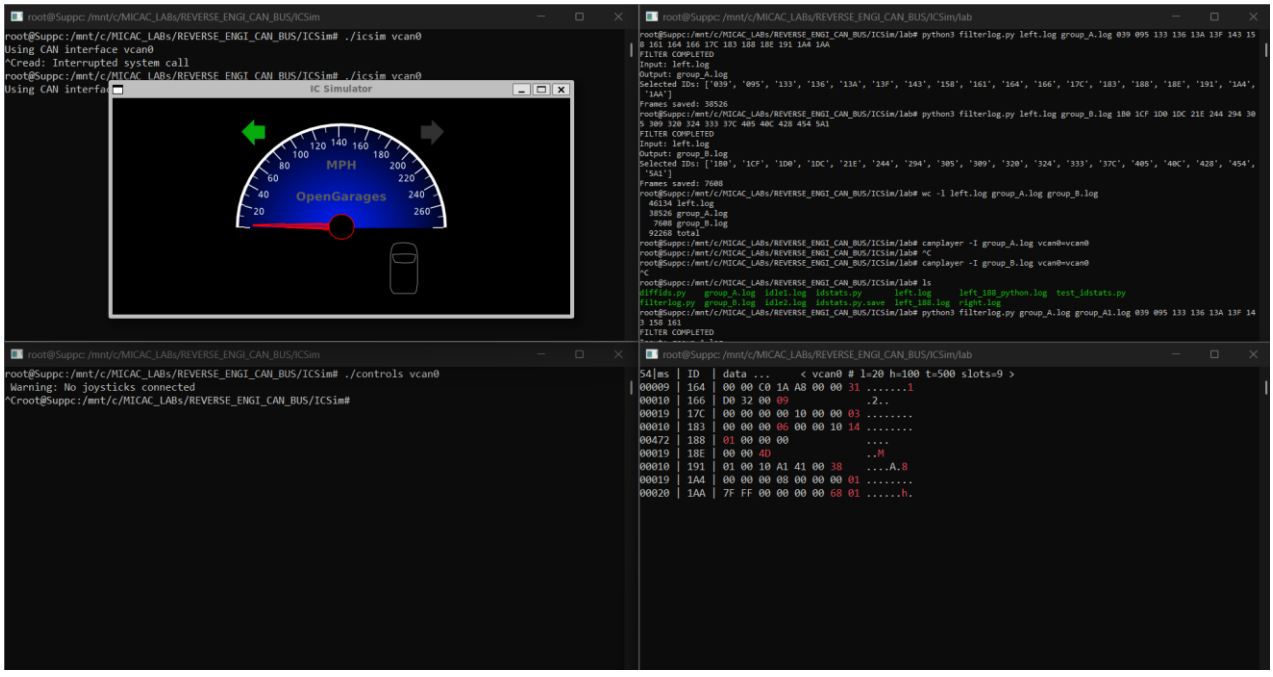


Figure 2. First bisection round: group A replay (38,526 frames), green left indicator visible; group B prepared separately

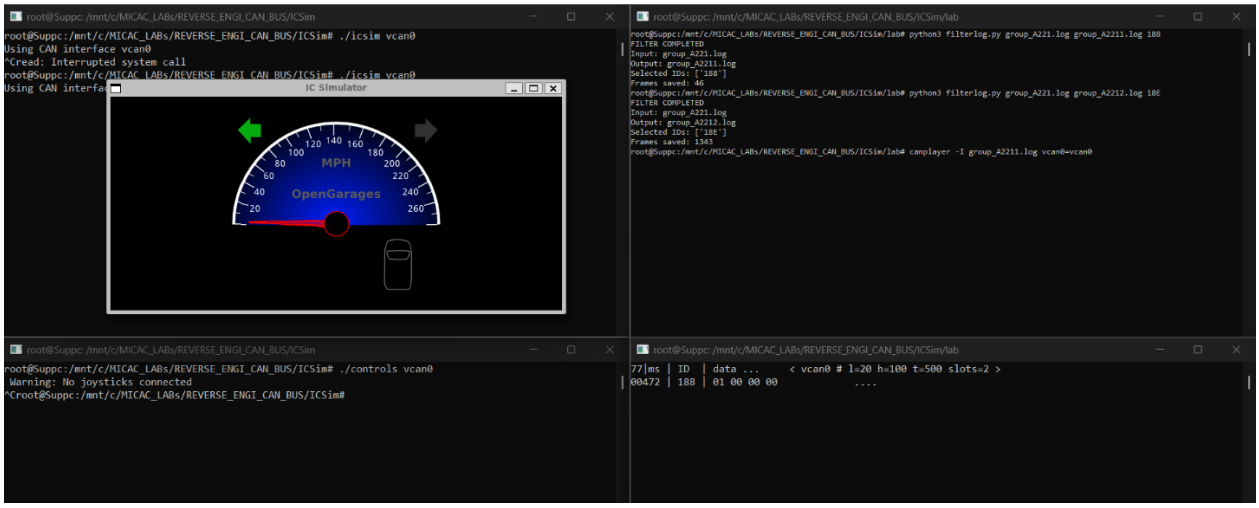


Figure 3. Final single-ID test: 0x188-only group_A2211.log (46 frames) and indicator activation in ICSim.

5. Byte-level decoding + remaining tasks

Once we get the desired ID, we can analyze the payload when the turn signal is off and when it's on – then compare them and try to figure out the necessary bit values to understand how the turn signal is controlled at the bit level.

Selected injection commands

```
cansend vcan0 188#01000000 #left
cansend vcan0 188#02000000 #righty
cansend vcan0 188#03000000 #lboth
cansend vcan0 188#00000000 #off
... ... ...
```

Once we get the desired ID, we can analyze the payload when the turn signal is off and when it's on – then compare them and try to figure out the necessary bit values to understand how the turn signal is controlled at the bit level.

First byte	Binary	Observed indicator state
0x00	000	Neither / off
0x01 / 0x05	001 / 101	Left
0x02 / 0x06	010 / 110	Right
0x03 / 0x07	011 / 111	Both
0x04	100	Neither / off

The roles of the remaining bits weren't determined during selection and analysis.

After the analysis and fine-tuning process of the algorithm have come to an end, it was decided to shift focus to more interesting targets for the "attacker," such as: Opening doors (to steal a car) or controlling the car's speed (to either lock the car in place or, on the contrary, create an accident situation and threaten the driver's life).

Speedometer: CAN ID 0x244

```
candump -L vcan0 > speed.log
python3 diffids.py idle1.log speed.log
grep ' 244#' speed.log | sed -n '400,430p'
cansend vcan0 244#0000001000
cansend vcan0 244#0000002000
cansend vcan0 244#0000003000
```

CAN payload (5 bytes)	Raw, bytes 3-4 (BE candidate)	Dashboard response
0000000000	0	0 MPH
0000001000	4096	About 30 MPH
0000002000	8192	About 60 MPH
0000003000	12288	About 80-90 MPH

The injection confirmed that ID 0x244 changes the speedometer readings. Bytes 3–4 are a plausible 16-bit signal in big-endian format, but the exact encoding, scale, and conversion to km/h haven't been fully adjusted; the upper gauge readings are just a visual estimate.

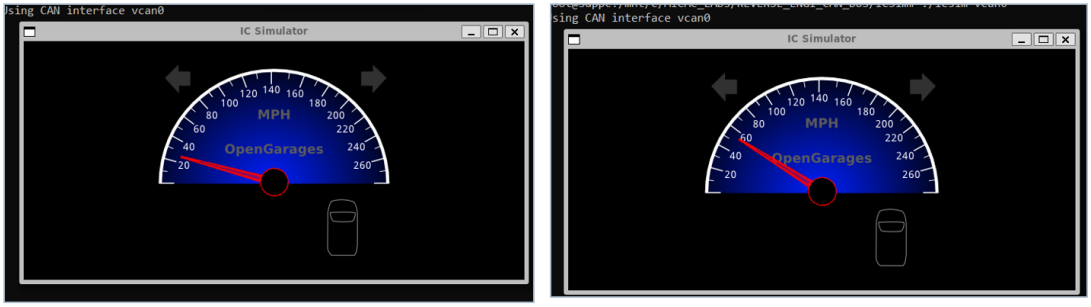


Figure 4. ICSim speedometer responses after selected 0x244 frame injections

Door locking: CAN ID 0x19B

```
candump -L vcan0 > doors.log
grep ' 19B#' doors.log
python3 diffids.py idle1.log doors.log
cansend vcan0 19B#00000F000000
cansend vcan0 19B#00000E000000
cansend vcan0 19B#000000000000
```

Value	Interpretation	Observation
0x0F (1111)	All doors are locked	All locked
0x0E (1110)	Door 1 unlocked; 2-4 locked	Door 1 unlocked
0x00 (0000)	All four doors are unlocked	All unlocked

The bit assignment are Door 1 = bit 0, Door 2 = bit 1, Door – 3 = bit 2 and Door 4 = bit 3. These three injected states all produced the expected visible results.

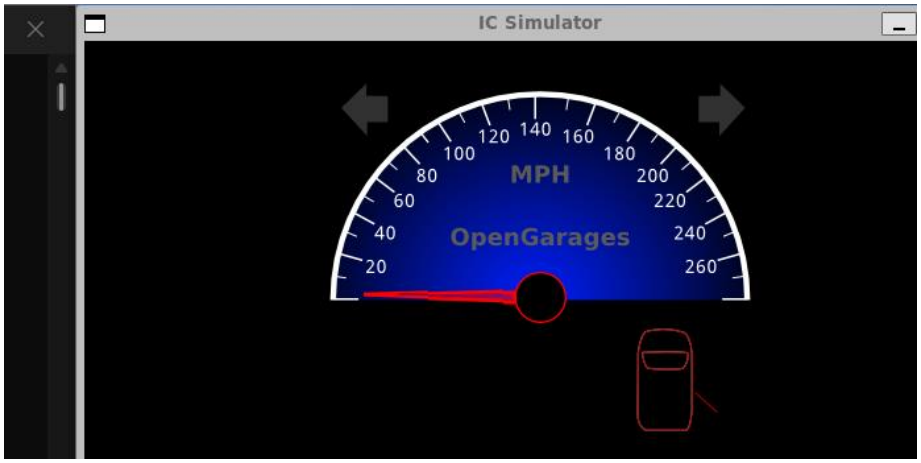


Figure 5. Door-status

6. Conclusions and Limitations

The experiment combined passive CAN traffic interception, differential analysis, a bisecting repeat-based approach, and active frame injection. In the idle recordings, 36 CAN IDs were captured; the two IDs with the most variable data were 0x244 and 0x183. Initially, comparing the idle recordings made it easier to distinguish normal changes from stimulus-related traffic. Bisecting isolated ID 0x188, and a repeat run just for 0x188 (46 frames) activated the left turn signal.

Controlled injections identified the flag bits for the left and right indicators (0x188), a numeric field related to the speedometer (0x244), and the four-door lock bit field (0x19B). These results show why receiving unauthorized CAN frames could let you manipulate the state. The findings relate to ICSim/vcan0, not a production car network. Speed scaling is still approximate, and handling of CAN counters or cryptographic protection wasn't implemented or tested here.